Event Sourcing & CQRS

Season: 4 · Enterprise · Category: Enterprise · Level: Intermediate

Intent

- Persist events; separate write/read models.

Problem

- Without this pattern, code tends toward: New things up with conditionals and duplicated construction.

Real-world Analogy

- Bank ledger vs balance view.

When to Use

- Microservices / modular monoliths.
- The design force named in Intent shows up more than once.

When Not to Use

- Greenfield CRUD without distribution.
- A simpler language feature (record, sealed switch, function value) already solves it.

Pattern Structure

- See mermaid diagram in explanation.md

Classic Implementation

```java
package com.buildwithwaqar.patterns.enterprise.eventsourcing;

import java.util.ArrayList;
import java.util.List;

sealed interface AccountEvent permits Deposited, Withdrawn {}
record Deposited(int amount) implements AccountEvent {}
record Withdrawn(int amount) implements AccountEvent {}

/** Minimal event-sourced aggregate + separate read model (CQRS). */
public final class Account {
    private final List<AccountEvent> events = new ArrayList<>();
    private int balance;

    public void deposit(int amount) {
        apply(new Deposited(amount));
    }

    public void withdraw(int amount) {
        if (amount > balance) throw new IllegalStateException("insufficient");
        apply(new Withdrawn(amount));
    }

    private void apply(AccountEvent e) {
        switch (e) {
            case Deposited(var a) -> balance += a;
            case Withdrawn(var a) -> balance -= a;
        }
```

Classic Implementation (2)

```
        events.add(e);
    }

    public int balance() { return balance; }
    public List<AccountEvent> events() { return List.copyOf(events); }
}

/** Read model projected from events. */
final class BalanceView {
    private int balance;
    void project(AccountEvent e) {
        switch (e) {
            case Deposited(var a) -> balance += a;
            case Withdrawn(var a) -> balance -= a;
        }
    }
    int balance() { return balance; }
}
```

Modern Java 25 / Functional

- Sealed events + pattern matching for projections.

Worked Example

```
See the JUnit test beside the implementation.
Run: mvn -Dtest=*Test test
Focus package matching: enterprise/eventsourcing/
```

Pitfalls

- Keep demos minimal — production needs idempotency and observability.
- Saga compensations must be careful.

Interview Q&A

- Q: Production concern for Event Sourcing & CQRS? A: Idempotency, observability, and failure modes.
- Q: How would you test this? A: Prefer injecting interfaces or interpreters; keep demos deterministic (fixed clocks, canned inputs).

Key Takeaways

- Persist events; separate write/read models.
- Prefer Java 25 sealed types / records / virtual threads where they simplify the classic shape.
- Ship working tests with every episode.

Video Script Highlights

- [0:00] Hook — Open with the analogy: Bank ledger vs balance view. [0:40] Intent — State the design force in one sentence: Persist events;...